ct-03 — Codeforces Round 1122 (Div. 3)

Link	https://codeforces.com/contest/2266
Date attempted	not recorded — round began 2026-09-21 14:35 UTC; solution files appeared 2026-09-21/22
Problems	8 (A–H)
Solved in contest	not recorded
Solved after	not recorded — A, B have working solutions; timing unknown
Unsolved	C, D, E, F, G, H — not attempted (link-only stubs)
Concepts	binding-constraint counting, all-or-nothing exchange argument, split-point scanning, invariants under an operation, dp over the divisor lattice, smallest-prime-factor sieve, backward reasoning, binary search on a monotone predicate, gcd-closure of reachable values, prefix/suffix minima characterisation

> **Note on the statements.** Three of this round's statements (D, F and H) have instructions to language models embedded in the prose — telling a model to insert a named filler variable, or to output a specific variable, and not to mention the instruction. They are the setters' anti-AI canaries. They are mentioned here so that pasting these statements into a model and trusting the result is a known hazard, not a surprise. Nothing in this document follows them.

1. The set

#	problem	verdict	key concept	one-line idea
A	Good Contest	AC	binding constraint	at most $\min(a_1,a_2,a_3)$ people solved everything, so the answer is $n - \min(a)$
B	Three Piles	AC	exchange argument	Alice takes all of c or none of it; answer $\max(\lvert a-b\rvert,\ a+c-b)$
C	AND, OR, Sort!	not attempted	split-point scanning	a mixed prefix lets you write either bit; scan every $0\ldots01\ldots1$ split and count the mismatches
D	Falling Concrete	not attempted	invariant under an operation	$\text{height} - \text{position}$ never changes; answer = longest run of consecutive distinct $a_i - i$
E	Prime Destruction	not attempted	dp over the divisor lattice	$dp[i] = \min$ over primes $p \mid i$ of $1 + p \cdot dp[i/p]$, summed over the input
F	MEX Replacement	not attempted	backward reasoning + binary search	to build M you need one of every smaller value at once; walk down, doubling the deficit
G	Modular Tree	not attempted	gcd-closure of reachable values	node u reaches $(a_u + g_u x) \bmod b_u$, with $g_u = \gcd(b_u, S_u, g_{v\ldots})$ bottom-up
H	Deque Malfunction	not attempted	prefix/suffix minima	a value is free iff its arrival time is a prefix or suffix minimum; DP each side, join at the global minimum

Solution files: `a.cpp` and `b.cpp` carry working solutions. `c.cpp` … `h.cpp` contain only the problem link — problems not attempted.

Two conventions notes. `a.cpp` and `b.cpp` are missing the `Problem link:` header that `CONVENTIONS.md` §2 requires (the stubs `c.cpp` – `h.cpp` have it), and the directory is named `cf-1122` after the round number while the URLs use contest id `2266` — both are the same contest, recorded here so the mapping is not lost.

Every algorithm written down in §3 was implemented and run against the official samples before being documented. H's characterisation was additionally validated by exhaustively enumerating which occurrence of each value to keep, on all nine samples.

---

2. Key concepts across the set

2.1 Reading the answer off the binding constraint — used in A

What it is: a counting problem where the answer is forced by whichever single constraint is tightest, and the construction that attains it is "make every constraint bind on the same objects".

How it works: the number of participants who solved all three problems is at most a_i for each i, hence at most $\min(a_1,a_2,a_3)$. That bound is attained: let the *same* $\min(a)$ participants solve all three, then hand out the remaining $a_i - \min(a)$ solves of each problem to anyone else, in any pattern. Upper bound plus matching construction — that pair is the whole proof, and skipping the construction half is how people talk themselves into a wrong bound.

Complexity: $O(1)$ per test.

How it showed up here: answer $= n - \min(a_1,a_2,a_3)$. `a.cpp` computes $\max$ over i of $(n - a_i)$, which is the same number — $\max(n - a_i) = n - \min(a_i)$.

The recognisable signal: "minimum number of X" where X is the complement of something bounded independently by several inputs. Bound the complement, then build the witness.

2.2 All-or-nothing: an exchange argument on a single resource — used in B

What it is: a two-player game where one contested resource is drawn from freely. The useful structural fact is that intermediate splits are never optimal for the player who moves first — the payoff is convex in how much they take, so the maximum sits at an endpoint.

How it works: the score is $|A - B|$, an absolute value, hence convex in the number of stones Alice takes. A convex function on an interval attains its maximum at an endpoint, so Alice's best first move is "take all of c" or "take none". Then the case analysis is short:

- $a \geq b$: Alice takes everything. Bob has nothing left to close the gap with, so the score is $a + c - b$.
- $a < b$: taking none leaves $b - a$, and Bob will not take stones because doing so widens the gap he is trying to close. Taking all gives $|a + c - b|$. Alice picks the better, and the result collapses to $\max(b - a, a + c - b)$.

Both cases are $\max(|a - b|, a + c - b)$; the second term can be negative, and the $\max$ against a non-negative first term absorbs that.

Complexity: $O(1)$ per test.

How it showed up here: `b.cpp` computes $\max(|a + c - b|, |a - b|)$, which is equivalent. When $a + c - b \geq 0$ the two expressions are identical; when $a + c - b < 0$ we must have $a < b$, so $|a + c - b| = b - a - c \leq b - a = |a - b|$ and both forms return $|a - b|$. Checked on the samples and on 2000 random triples — no disagreement.

The recognisable signal: a min/max game over "how much of a pile do I take", with an objective that is convex (an absolute value, a maximum, a distance). Convexity means you only ever have to test the endpoints.

Sources:

- USACO Guide — Greedy Algorithms with Sorting — exchange arguments as the standard correctness tool

2.3 Split-point scanning with prefix and suffix counts — used in C

What it is: when the target is a *shape* rather than a specific string — here "sorted", i.e. `0…01…1` — there are only `n+1` targets, one per split point. Enumerate them and cost each in `O(1)`.

How it works: maintain `onesLeft[i]` = number of `1`s in `s[0..i)` and `zerosRight[i]` = number of `0`s in `s[i..n)`. The cost of the split at `i` is `onesLeft[i] + zerosRight[i]` — every character that disagrees with the target must be rewritten, and (see below) each such rewrite is exactly one operation. Take the minimum over all `i`.

Complexity: `O(n)` time, `O(n)` space (or `O(1)` with one forward pass keeping a running count).

How it showed up here: the enabling observation comes first, and it is the actual difficulty of the problem: replacing s_i with the AND or the OR of $s_1 \ldots s_i$ lets you write **either** bit into position `i` — but only if that prefix already contains both a `0` and a `1`, because AND and OR of a uniform prefix both reproduce the prefix's own value. Two consequences:

- s_1 can never change (its prefix is itself). So if $s_1 = 1$, the string must end up all ones, and the answer is simply the number of zeros.
- If $s_1 = 0$, order the work so each rewrite is legal: first flip the zeros that need to become ones on the right (using OR), then the ones that need to become zeros on the left (using AND). A split placed inside the leading block of zeros is never better than sliding it to the end of that block.

The recognisable signal: "make the array sorted / monotone / of shape P" with `P` parameterised by a single cut. Enumerate the cut.

Sources:

- Codeforces — Round 1122 (Div. 3) editorial

2.4 Invariants under an operation — used in D

What it is: the single highest-leverage habit for constructive problems. When an operation is described concretely ("move this element there and adjust these values"), the first question is not "what can I build" but **"what does this operation never change?"** The invariant collapses an unbounded search space into a static property of the input.

How it works: in D, moving section `j` back to position `i` does two things: the carried section travels `j − i` positions left and loses exactly `j − i` height; every section it passes over moves one position right and gains exactly `1` height. In both cases `height − position` is unchanged. So the multiset $\{a_i - i\}$ is invariant across *any* sequence of operations, and each section carries its own value forever.

Then the characterisation, in both directions:

- **Necessary:** if `k` consecutive positions starting at `p` all have height `h`, their invariant values are `h−p, h−p−1, …, h−p−k+1` — `k` *distinct consecutive* integers.
- **Sufficient:** if `k` sections have consecutive invariant values, placing them in decreasing order of value makes their heights equal. And the operation is powerful enough to realise any order, by building the target arrangement left to right and dragging the next needed section into place.

So the answer is the **longest run of consecutive integers among the distinct values** $a_i - i$. Duplicates are not just deduplicated for convenience — two sections sharing an invariant value can never sit in the same flat block, because a flat block needs distinct values.

Complexity: `O(n log n)` with a sorted set, `O(n)` with hashing.

The recognisable signal: an operation that shifts positions while adjusting values by the same amount it shifts. Compute the operation's effect on `value ± index` and one of the two will fall out constant.

Sources:

- Codeforces — Round 1122 (Div. 3) editorial

2.5 DP over the divisor lattice, and the smallest-prime-factor sieve — used in E

What it is: a DP whose states are the integers `1..n` and whose transitions follow division by a prime, so every edge strictly decreases the value and the order of evaluation is just "increasing `i`".

How it works: let `dp[i]` be the minimum number of operations to reduce a single copy of `i` to elements all at most `k`. Then `dp[i] = 0` for `i ≤ k`, and for `i > k`:

```
dp[i] = min over primes p | i of ( 1 + p · dp[i/p] )
```

The `p ·` factor is the part that is easy to drop: one operation on `i` produces **`p` copies** of `i/p`, and each of those must then be destroyed independently, so the sub-cost is paid `p` times, not once. Because the elements never interact, the answer for the whole multiset is Σ `dp[`a_i`]`.

Enumerating the *distinct* prime divisors of every `i` is what a smallest-prime- factor table gives you: store `lp[i]`, then peel `i` by repeatedly dividing out `lp[i]`. CP-Algorithms' linear sieve computes `lp` for `2..n` in `O(n)`.

Complexity: `O(n log log n)` per test after the sieve (CP-Algorithms' linear variant makes the sieve itself `O(n)`), with `O(log i)` distinct primes per value.

Overflow is real here. With `k = 1`, `dp[2^m] = 2^m − 1`, so at `n = 2×10^5` a single element can cost ~`1.3×10^5` operations and the sum over `2×10^5` elements reaches ~`2.6×10^{10}`. The answer needs 64 bits even though every input value fits in 16.

The recognisable signal: an operation that replaces a value by several copies of a divisor, with values bounded by `n`. The "several copies" is the multiplier in the recurrence.

Sources:

- CP-Algorithms — Linear Sieve — computes the minimum prime factor of every number in `[2, n]` in `O(n)`
- Codeforces — Round 1122 (Div. 3) editorial

2.6 Backward reasoning plus binary search on a monotone predicate — used in F

What it is: two techniques that fit together. First, when an operation *builds* a value out of a set of prerequisites, reason from the goal backwards and count what it demands. Second, when feasibility is monotone in the target, binary search the target instead of computing it.

How it works: the operation replaces a chosen sub-collection by its MEX, so producing `M` requires one copy of every value `0, 1, …, M−1` present simultaneously. Walk **downwards** from `M−1`, carrying `q` = how many copies of the current value the construction still needs:

- `c ≥ q` (enough copies on hand): consume `q`, and record `c − q` as *extras*.
- `c < q`: the missing `q − c` copies must each be manufactured, and manufacturing one copy of `x` itself demands one copy of every value below `x`. So `q ← q + (q − c)`.

In one step: `q ← q + max(0, q − c)`. A run of `d` consecutive **absent** values doubles `q` each time, i.e. multiplies it by `2^d` — so cap `q` as soon as it exceeds the total element count, or it overflows. At the bottom, the check succeeds iff `zeros + extras ≥ q`: leftover positive elements cannot help at another positive level, but any single element can be turned into a `0` by taking it alone (its MEX is `0` unless it *is* `0`).

Feasibility is monotone — building `M` requires every smaller value to exist along the way — so binary search applies. The search range is tiny: past roughly `max(xᵢ) + 50`, the gaps guarantee enough doublings that no input of size `2×10^5` can pay for it.

Complexity: `O(n log n)` to sort plus `O(n)` per check, `O(n log n)` overall.

The recognisable signal: "maximum value ever reachable" under an operation that consumes a set and emits one element. Forward search over multisets is hopeless; backward accounting from a single target is linear.

Sources:

- USACO Guide — Binary Search — the monotonicity precondition
- Codeforces — Round 1122 (Div. 3) editorial

2.7 gcd-closure of reachable values — used in G

What it is: characterising the full set of values a node can hold, rather than searching for the best one. The set turns out to be an arithmetic progression — a coset of the subgroup generated by a gcd — which is why it is describable by a single number per node.

How it works: for node u, the reachable values are exactly $(a_u + g_u \cdot x) \bmod b_u$ over integers x, for one modulus-like number g_u computed bottom-up. The operation adds the current children-sum S to x_u mod b_u; doing it repeatedly adds multiples of S, and varying a child v by multiples of g_v varies what gets added by multiples of g_v. Everything reachable is therefore an integer combination, and the set of integer combinations of a collection of numbers is exactly the multiples of their gcd:

$$g_u = \gcd(\, b_u,\ S_u,\ \{\, g_v : v \text{ a child of } u \text{ with } g_v \neq b_v \,\}\,)$$

where S_u is the sum of the children's *initial* values. A child with $g_v = b_v$ is frozen at a_v (its progression has step b_v, which is 0 mod b_v) and contributes nothing. A leaf gets $g_u = b_u$, i.e. frozen — correct, since a leaf has no children to sum.

Given g_u, the largest reachable value is the largest element of the progression below b_u:

$$a_u + \operatorname{floor}\big((b_u - 1 - a_u) / g_u\big) \cdot g_u$$

The last step is that these per-node maxima are **simultaneously** achievable, so the answer is their plain sum: set a node to its maximum, then do all remaining work strictly inside its children's subtrees, which never disturbs it. Work top-down when actually performing the operations.

Complexity: `O(n log(10^9))` — one DFS, with a gcd at each node.

The recognisable signal: repeated additive updates under a modulus. The reachable set is never "everything" and never "just the start"; it is the multiples of a gcd, and finding which gcd is the problem.

Sources:

- cppreference — std::gcd — `<numeric>`, C++17, `gcd(24, 0) == 24` (the identity that makes the $S_u = 0$ case fall out correctly)
- Codeforces — Round 1122 (Div. 3) editorial

2.8 Prefix and suffix minima as a feasibility characterisation — used in H

What it is: replacing "can this be built without a special move?" with a static, checkable property of the chosen arrival times, and then optimising over the choices.

How it works: first, throw away the noise — any occurrence you do not intend to keep can be pushed to an end and immediately popped off that same end, free. So the only decision is **which occurrence of each value `1..n` to keep**. Let t_i be the arrival index of the kept occurrence of value `i`.

Value `i` can be pushed to the **front** without a malfunction exactly when everything already in the deque belongs to the right of `i` in the final arrangement — that is, every kept value smaller than `i` arrives *later*: $t_i < \min(t_1, \ldots, t_{i-1})$. Symmetrically it can be pushed to the **back** exactly when $t_i < \min(t_{i+1}, \ldots, t_n)$. So:

> a value avoids a malfunction ⟺ its t_i is a **prefix minimum** or a **suffix minimum** of the sequence $t_1 \ldots t_n$.

The global minimum is both, so with `L` prefix minima and `R` suffix minima, `L + R − 1` values are free and the answer is `n − max(L + R − 1)`. Fixing the position of the global minimum makes the two sides independent, so they can be optimised separately and joined.

Each side is a DP over arrival positions: `dp[p]` = most prefix minima achievable so far when the smallest kept arrival position is `p`. For an occurrence at `t`, making it the new minimum gives `1 + max_{p > t} dp[p]`; and after processing value `i`, every state with `p` beyond `i`'s last occurrence must be discarded, since `i` would have no occurrence left to place after it. A lazy segment tree does the range max and the suffix clear in `O(log m)`.

Complexity: `O(m log m)` per test.

The recognisable signal: a deque or two-ended structure where an element is "free" only when it is extremal among what remains. Prefix/suffix extrema are the usual closed form, and they usually decompose at the global extremum.

Sources:

- CP-Algorithms — Segment Tree
- Codeforces — Round 1122 (Div. 3) editorial

2.9 `std::abs` and 64-bit hygiene — used in B

What it is: `std::abs` has overloads in two different headers — the integral ones come from `<cstdlib>` (and `<cmath>` in libstdc++ as an extension), the floating-point ones from `<cmath>`. Including only `<cmath>` and calling `std::abs` on a `long long` is a portability hazard: where the integral overload is not visible, the call resolves to the `double` version and silently truncates above `2^53`.

How it showed up here: `b.cpp` includes `<cmath>` and `<iostream>` and calls `std::abs` on `long long`. It works on the Codeforces toolchain, and the values here (`≤ 2×10^9`) are exactly representable as `double` anyway, so nothing is wrong with the submission. The habit is what matters: include `<cstdlib>` (or use `std::llabs`) when taking absolute values of 64-bit integers.

The 64-bit choice itself is correct and necessary — `a + c` reaches `2×10^9`, past `INT_MAX`, and the statement says so explicitly. `a.cpp` gets away with `int` because `n ≤ 9` there.

Sources:

- cppreference — std::gcd (for the general `<numeric>` / `<cstdlib>` header discipline)

---

3. From brute force to optimal, problem by problem

A — Good Contest

- **Rung 1 — enumerate scoreboards.** Each of `n ≤ 9` participants has one of `8` solve-patterns: `8^9 ≈ 1.3×10^8` per test, times `t ≤ 3000`. Correct, far too slow, and it is the definition.
- **Rung 2 — bound the complement.** Count the *strong* participants instead of the weak ones: at most $\min(a_1, a_2, a_3)$, attained by the same people solving all three. Answer $n - \min(a)$, $O(1)$.

`a.cpp` is rung 2, written as $\max_i (n - a_i)$. Identical value. The `ans = 0` initialiser is dead — $a_i \le n$ is guaranteed, so every term is already non-negative — but harmless. `<cmath>` is included where `std::max` actually lives in `<algorithm>`; it compiles through transitive includes, the same nit as C's missing `<vector>` in ABC 476.

B — Three Piles

- **Rung 1 — game tree.** Recurse on `(a, b, c, whose turn)` and memoise. `c ≤ 10^9` makes the state space impossible, but on small `c` it is the oracle that confirms the closed form.
- **Rung 2 — Alice takes all or nothing.** $|A - B|$ is convex in how much Alice takes, so the optimum is at an endpoint (§2.2). That removes the continuum.
- **Rung 3 — case split to a closed form.** $a \ge b$: take everything, score $a + c - b$. $a < b$: $\max(b - a, a + c - b)$, because Bob never takes stones when he is already ahead. Both collapse to $\max(|a - b|, a + c - b)$, $O(1)$.

`b.cpp` is rung 3 in the equivalent form $\max(|a + c - b|, |a - b|)$ — see §2.2 for why the two agree, verified on the samples and 2000 random triples. `long long` is correctly used. See §2.9 for the `<cmath>` / `std::abs` habit.

C — AND, OR, Sort!

- **Rung 1 — search the state space.** BFS over all `2^n` strings. `n ≤ 2×10^5`.
- **Rung 2 — characterise one operation.** The real work. AND and OR over a prefix containing both bits give `0` and `1` respectively, so position `i` becomes freely writable — one operation, either value — as long as $s_1 \ldots s_i$ is mixed. A uniform prefix reproduces its own value, so s_1 is immutable forever.
- **Rung 3 — enumerate the target.** A sorted binary string is `0^i 1^{n−i}`. If $s_1 = 1$ the only reachable sorted string is all-ones, costing `count('0')`. Otherwise scan every split `i` and cost it as `onesLeft[i] + zerosRight[i]`, taking the minimum. Ordering the rewrites (OR on the right first, then AND on the left) keeps every one of them legal. `O(n)`.

Where the difficulty actually sits: rung 2, not rung 3. The scan is routine once you know each mismatched position costs exactly one operation; the earned insight is that "AND or OR of a mixed prefix" is a *write-anything* primitive. The community difficulty vote splits 29 "average" / 27 "hard", which matches: it is an easy implementation guarding a non-obvious observation.

D — Falling Concrete

- **Rung 1 — search over operation sequences.** Unbounded; there is no state space to enumerate.
- **Rung 2 — ask what the operation preserves (§2.4).** The carried section moves `j − i` left and loses `j − i` height; every passed-over section moves `1` right and gains `1`. So `height − position` is invariant, for every section, forever.
- **Rung 3 — characterise flatness in invariant space.** `k` consecutive equal heights ⟺ `k` *distinct consecutive* invariant values, in both directions; and any ordering of the sections is reachable, so there is no extra constraint.
- **Rung 4 — compute it.** Put $\{a_i - i\}$ in a `std::set` (dedupe is not an optimisation — two sections with the same invariant can never share a flat block) and find the longest run of consecutive integers. `O(n log n)`.

This was the round's wall: the difficulty vote is 85 "hard" and 9 "impossible" against 4 "easy", and it is also the best-liked problem (57 "excellent"). The whole distance between rung 1 and rung 4 is one question — *what is conserved?* — and the code that follows is ten lines.

Verification: checked against all eight official samples.

E — Prime Destruction

- **Rung 1 — simulate the multiset.** Apply operations and search. The multiset grows multiplicatively; hopeless.
- **Rung 2 — elements are independent.** Nothing couples the elements, so the total is `Σ` of a per-value cost, and the problem shrinks to "destroy one `i`".
- **Rung 3 — the recurrence.** `dp[i] = 0` for `i ≤ k`; otherwise `dp[i] = min over primes p | i of (1 + p·dp[i/p])` — the `p·` because one operation yields `p` copies that each pay the sub-cost (§2.5). Evaluate in increasing `i`, since `i/p < i`.
- **Rung 4 — get the primes cheaply.** A smallest-prime-factor sieve over `1..n` gives the distinct prime divisors of every value by repeated division, `O(n)` with the linear sieve. Answer `Σ dp[aᵢ]`, in `long long`.

This is the most *standard* problem in the set — sieve plus a DP over values, no insight spike — which makes it the best next target after A and B, ahead of C and D. The editorial's bonus (compute `f(k)` for **all** `k` simultaneously, which was the original version) is the natural follow-up.

Verification: checked against all six official samples.

F — MEX Replacement

- **Rung 1 — forward search.** BFS over reachable multisets. Astronomical.
- **Rung 2 — reason backwards from one target.** To have `M` appear you need one copy of each of `0..M−1` at the same moment. Walk down from `M−1` keeping `q` = copies still required.
- **Rung 3 — the deficit recurrence.** `q ← q + max(0, q − c)` where `c` is the supply of the current value, accumulating unused positives as *extras*; at the bottom, succeed iff `zeros + extras ≥ q`, because any leftover element can be converted to a `0` on its own. A run of `d` missing values multiplies `q` by `2^d`, so apply it in one shot and cap `q` at the total element count to avoid overflow.
- **Rung 4 — binary search the target.** Feasibility is monotone (building `M` forces every smaller value to exist en route), so binary search `M`. The editorial's bonus tightens the range to about `max(xᵢ) + 50`: past that, the consecutive gaps double `q` faster than `n ≤ 2×10^5` elements can pay for. `O(n log n)`.

Verification: implemented with the `max(xᵢ) + 50`-style bound and checked against all ten official samples.

G — Modular Tree

- **Rung 1 — search.** Operations can be applied in any order, any number of times; no finite state space to walk.
- **Rung 2 — ask what one node can reach, not what to do.** For each `u` the reachable set is a progression `(a_u + g_u x) mod b_u` (§2.7). This is the reframe that makes the problem finite.
- **Rung 3 — compute `g_u` bottom-up.** `g_u = gcd(b_u, S_u, {g_v : v child, g_v ≠ b_v})`, with `S_u` the sum of children's initial values. Leaves get `g_u = b_u` and are frozen.
- **Rung 4 — maxima are independent, so just sum them.** Each node's best is `a_u + floor((b_u − 1 − a_u)/g_u)·g_u`; fixing a node at its maximum and then working only inside its children's subtrees leaves it untouched, so all maxima hold at once. One DFS, `O(n log(10^9))`.

Verification: checked against all eight official samples, including the `10^9`-modulus case that answers `3999999996`.

H — Deque Malfunction

- **Rung 1 — brute force the choices.** Pick which occurrence of each value to keep, then count prefix/suffix minima. The product of the occurrence counts — fine on the samples (this is how the characterisation below was validated), exponential in general.
- **Rung 2 — discard the extras for free.** Any occurrence you do not keep can be pushed to an end and popped from that same end, costing nothing. So the problem is purely "which occurrence of each value do I keep?". Missing value ⟹ `-1`.
- **Rung 3 — the characterisation (§2.8).** With `t_i` the kept arrival index of value `i`, value `i` is free ⟺ `t_i` is a prefix minimum or a suffix minimum of `t_1…t_n`. With `L` prefix minima and `R` suffix minima, `L + R − 1` are free.
- **Rung 4 — DP each side, join at the global minimum.** `dp[p]` = best count of prefix minima when the running minimum sits at arrival position `p`; an occurrence at `t` transitions to `1 + max_{p>t} dp[p]`; after each value, clear every state past that value's last occurrence. A lazy segment tree gives range max and suffix clear in `O(log m)` — or a plain segment tree plus a `std::set` of live positions, since each position dies at most once, which amortises to the same bound. Run it forwards for `L[p]`, backwards for `R[p]`, answer `n − max_p (L[p] + R[p] − 1)`. `O(m log m)`.

Verification: rung 3's characterisation was checked by exhaustive enumeration over occurrence choices on all nine official samples, including the `-1` case. Rung 4 is the editorial's and is reported, not independently implemented here.

---

4. Key ideas that would have helped

- **"What does this operation preserve?" would have unlocked D outright.** D is the problem the round was built around — hardest-voted and best-liked — and the entire solve is noticing that the carried section loses exactly as much height as the distance it travels, so `a_i − i` is fixed. Whenever an operation moves things and adjusts values in the same breath, compute its effect on `value + index` and `value − index` before anything else. That is a 60-second check with a very high hit rate.
- **E, not C or D, was the right third problem to spend time on.** It is a sieve plus a DP over values — technique you can prepare, with no insight spike. In a Div. 3 set the letters are not reliably ordered by *your* difficulty; a quick scan of all eight statements before committing to C would have surfaced it.
- **C is guarded by one observation, not by its implementation.** "AND or OR over a mixed prefix writes any bit you like" converts it into a split-point scan you could write in five minutes. If you were stuck on C, the thing to practise is asking *what a single operation can achieve in the best case*, before asking what sequence to use.
- **F and H are the same instruction in two costumes: reason backwards from the goal.** F counts what one copy of `M` demands; H asks which arrivals could possibly have been placed without help. Forward search is intractable in both; backward accounting is linear in both.
- **Two `O(1)`-formula problems means the round's real start was C.** A and B are closed forms; the set's difficulty steps up hard at C and again at D. Budget accordingly — finishing A and B quickly is expected, not ahead of schedule.
- **Add the `Problem link:` header to every solution file.** `a.cpp` and `b.cpp` are missing it (`CONVENTIONS.md` §2), which is precisely the header this write-up needs to fetch statements from. The stubs have it; the solved ones do not.

5. Reading the constraints

#	limits	what that was signalling
A	`t ≤ 3000`, `n ≤ 9`, `0 ≤ aᵢ ≤ n`	`n ≤ 9` is far too small to matter — a closed form is expected, and the tiny bound is there to make the counting argument obvious
B	`t ≤ 10^4`, `0 ≤ a,b,c ≤ 10^9`	`O(1)` per test. `a + c` exceeds `INT_MAX`; the statement says "use a 64-bit integer type" outright — a constraint note doubling as a hint
C	`Σn ≤ 2×10^5`, binary string	`O(n)` or `O(n log n)`. "Binary" plus "sorted" means the target has only `n+1` possible shapes — enumerate them
D	`Σn ≤ 2×10^5`, `n ≤ aᵢ ≤ 10^9`	`O(n log n)`. The odd lower bound `aᵢ ≥ n` exists only to guarantee heights stay positive — a hint that heights *decrease* by at most `n`, i.e. by the distance travelled
E	`Σn ≤ 2×10^5`, `1 ≤ aᵢ ≤ n`	`aᵢ ≤ n` is the whole signal: values are bounded by the array length, so a sieve and a `dp[]` indexed by value both fit. Answers need `long long`
F	`Σn ≤ 2×10^5`, `xᵢ ≤ 10^9`, `yᵢ ≤ 10^9`	values up to `10^9` but only `2×10^5` distinct ones ⟹ work on the compressed pairs. Multiplicities up to `10^9` mean the count arithmetic is 64-bit
G	`Σn ≤ 2×10^5`, `bᵢ ≤ 10^9`, `0 ≤ aᵢ < bᵢ`	`O(n log C)` — one DFS with a gcd per node. `aᵢ < bᵢ` says the values start already reduced, so no normalisation pass is needed
H	`Σm ≤ 2×10^5`, `n ≤ m`, `1 ≤ bᵢ ≤ n`	`O(m log m)` — a DP with a segment tree. `n ≤ m` allows duplicates, which is the entire problem: choosing *which* occurrence to keep

6. Post-mortem

Where the time went: not recorded.

What to drill:

1. **The invariant check** (§2.4), as a reflex rather than a technique: for any operation that moves elements and adjusts values, immediately evaluate its effect on `value − index` and `value + index`. D is the worked example; run it cold and time yourself.
2. **Sieve + DP over values** (§2.5). E is entirely this, and it is the most transferable single item in the round. Then attempt the editorial's bonus — all of `f(1) … f(n)` at once.
3. **Backward reasoning from a target** (§2.6, §2.8), on F and H. Both become tractable the moment you stop simulating forwards.
4. **C, first, as a five-minute exercise** once §2.3's observation is internalised — it is the cheapest real solve available in this set.

Open: C, D, E, F, G, H are all still unsolved in code — `c.cpp` … `h.cpp` are stubs. §3 gives a full route for each. Suggested order: **E → C → D → F → G → H**, which is roughly increasing in how much insight (as opposed to technique) each one demands, and deliberately not alphabetical.